

Bonus: REST Philosophy

 [_ \(https://github.com/learn-co-curriculum/python-p4-rest-philosophy\)](https://github.com/learn-co-curriculum/python-p4-rest-philosophy)  [_ \(https://github.com/learn-co-curriculum/python-p4-rest-philosophy/issues/new\)](https://github.com/learn-co-curriculum/python-p4-rest-philosophy/issues/new)

Learning Goals

- Build RESTful APIs that are easy to navigate and use in applications.
-

Key Vocab

- **Representational State Transfer (REST)**: a convention for developing applications that use HTTP in a consistent, human-readable, machine-readable way.
 - **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
 - **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
 - **GET** : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
 - **POST** : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
 - **PATCH** : an HTTP request method that signifies that the client is attempting to update a resource with new information.
 - **PUT** : an HTTP request method that signifies that the client is attempting to update a resource with new information contained in a complete record.
 - **DELETE** : an HTTP request method that signifies that the client is attempting to delete a resource.
-

Introduction

We've done a lot with REST in this module, but there's still much more to learn- Roy Fielding wrote his Ph.D. dissertation on it ([Architectural Styles and the Design of Network-based Software Architectures.pdf](https://www.ics.uci.edu/%7Efielding/pubs/dissertation/fielding_dissertation.pdf) , after all. You don't need a Ph.D. to understand REST, but some of the underlying ideas are a bit more complicated than you'll need to use in day-to-day programming.

That being said, if you're curious (or headed into a particularly intense interview), this lesson will provide you more context on the rationale behind REST and the core tenets of the philosophy.

6 Architectural Constraints

There are infinitely many things that you can do with code, and the number of possibilities grows even further when software applications start to interact with one another. Interfaces between software applications (APIs) can be built in many different ways, but there is always a learning curve when starting to interface with a new application.

RESTful APIs are held in such high esteem in the software industry because of their rigid, uniform standards that serve a wide variety of goals. You should not always use REST- sometimes you need to perform functions that don't fit into its standards- but you should aim to use it wherever it makes sense.

We know at this point that RESTful APIs are designed to take advantage of HTTP verbs, but there are actually five strict (and one less strict) architectural constraints that determine if an API is RESTful:

Uniform Interface

Resources inside of a RESTful API must be named and exposed consistently. A resource should have one and only one logical URL, and related data should be accessible using either a smaller URL contained within or a larger URL extended from its end.

Resources should not be too large; large resources, like our " **GET** all" resources from this module, should contain links to resources for individual records when those individual records aren't the purpose of the larger resource.

All resources should be accessible through HTTP methods (you know this one!) and in a consistent format such as JSON or XML.

Stateless

The server running a RESTful API should not keep track of the client's requests. Essentially, every request might as well be from an entirely unique entity.

NOTE: This means that for applications that require users to log in, the user's session information must be sent with every request.

Cacheable

Resources should be cacheable and information on caching should be shared with the client. Caching means that the client is storing the information from a request on its side- eliminating client-server interactions whenever possible. This speeds things up for users and reduces the load on your server. Neat!

Client-Server

The client and server should not need to maintain a constant connection in order to interact with one another. If the client accesses a resource, it should be able to predict the format of whatever's there based on the last request it sent to your API.

Layered System

The architecture of a RESTful API needs to be contain multiple layers. This could mean a database layer, an application layer, and a client- like we've built so far- or it could mean any number of intermediate servers for caching, load-balancing, and so on.

Each layer of a layered system architecture should know *nothing* about the other layers. Any information should be retrieved through new inter-layer communications.

Optional: Code-on-Demand

REST APIs can serve code snippets (even objects and scripts!) to the client. This is not always important functionality, so it is considered optional.

Conclusion

REST provides a set of architectural constraints that make APIs easy to build and understand. REST isn't always the best approach to building an API- some ultra-secure resources should not be cached, for instance- but when appropriate, it provides your partners and customers with a predictable set of behaviors and encourages them to start using your product.

Some of these constraints are a bit *academic*- they were proposed in a Ph.D. dissertation, after all! But do your best to adhere to these constraints when building RESTful APIs and trust your senior colleagues to help you out when you need it. It takes a while to get the hang of REST, but it's worth it!

Resources

- [What RESTful Actually Means](https://codewords.recurse.com/issues/five/what-restful-actually-means) ➞ <https://codewords.recurse.com/issues/five/what-restful-actually-means>
- [REST API Architectural Constraints](https://www.geeksforgeeks.org/rest-api-architectural-constraints/) ➞ <https://www.geeksforgeeks.org/rest-api-architectural-constraints/>